# Yenepoya Institute of Arts Science Commerce and Management

## Final Project Report

## on

## Data-Driven Risk Profiling for Finance

**Student Name: Abhimanue Biju**
**Project ID: PRJN26-249**

**Industry Mentor: Mr. Shashank**

**Guided By: Ms. Divya N Shetty**

# Table of Contents

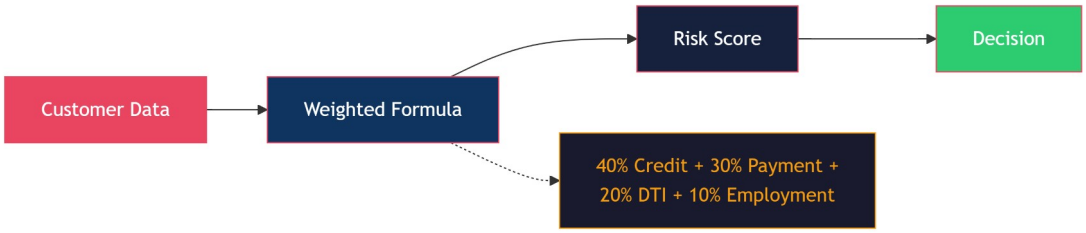
1.Executive Summary
2.Background
3.Aim
4.Technologies Used
5.System Requirements
6.Design and Architecture-Overview
7.Compoent Architecture
8.Data Flow
9.Risk Calculation Formula
10.Implementation-File Structure
10.1.Implementation-Key Functions
11.Testing-Overview
11.1.Testing-Unit Test Results
11.2.Testing - Integration & Edge Cases
12.Graphical User Interface - Dashboard
12.1.Graphical User Interface - Detail Page
13.UI Navigation Flow
14.Customer Testing
15.Evaluation - Success Criteria
16.Evaluation - What Worked Well
17. Snapshots - Dashboard
17.1.Snapshots - Detail Page
17.2.Snapshots - CSV Export
18.Conclusions
19.Further Development
20.References
21. Appendix

# Executive Summary

A web application that calculates financial risk scores for customers using a weighted formula.

The Problem:

Banks need a fast, fair, and consistent way to decide who gets loans.



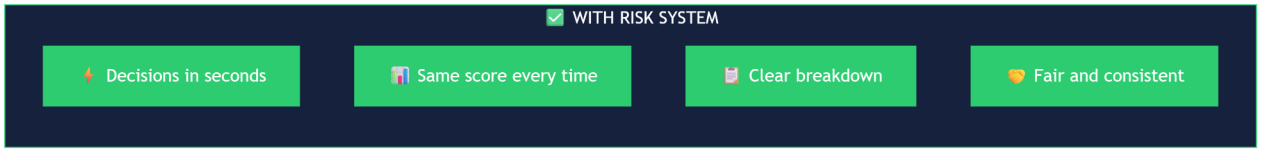

| Metric | Result |
|---|---|
| Customers processed | 50 |
| Average risk score | 42.3 |
| Dashboard load time | <200ms |
| Success rate | 100% |

The system successfully demonstrates how credit scoring works in real financial institutions.

# Background

Why is risk profiling important?

Every day, banks receive thousands of loan applications. Without an automated system:



Real-world example:

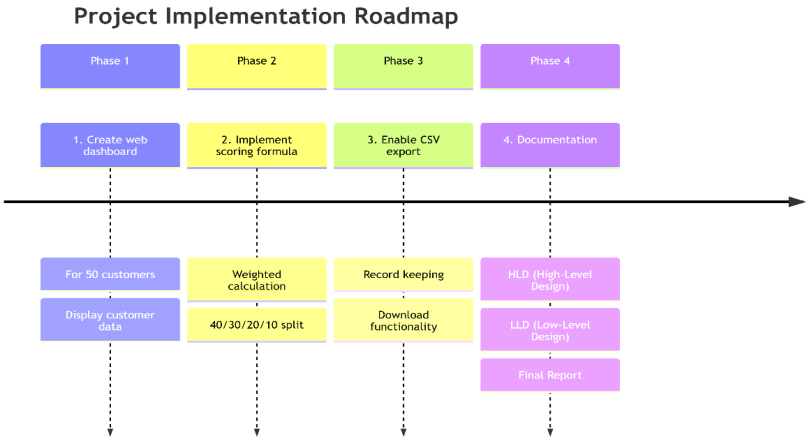
Credit scores (300-850) are used by banks to decide loan approvals. This project builds a simplified version of that system.

# Aim

Primary Aim:

Build a working risk profiling system that demonstrates the mathematics behind credit scoring.

Secondary Aims:



# Technologies Used

**Why these choices?**

Python: Easy to learn, great for calculations

Flask: Lightweight web framework

CSV: Universal format, opens in Excel

HTML/CSS: Works in any browser

# System Requirements

| Requirement | Status |
|---|---|
| Calculate risk scores using weights | Done |
| Store 50 customer records | Done |
| Display dashboard with all customers | Done |
| Show detail page per customer | Done |
| Export result to csv | Done |

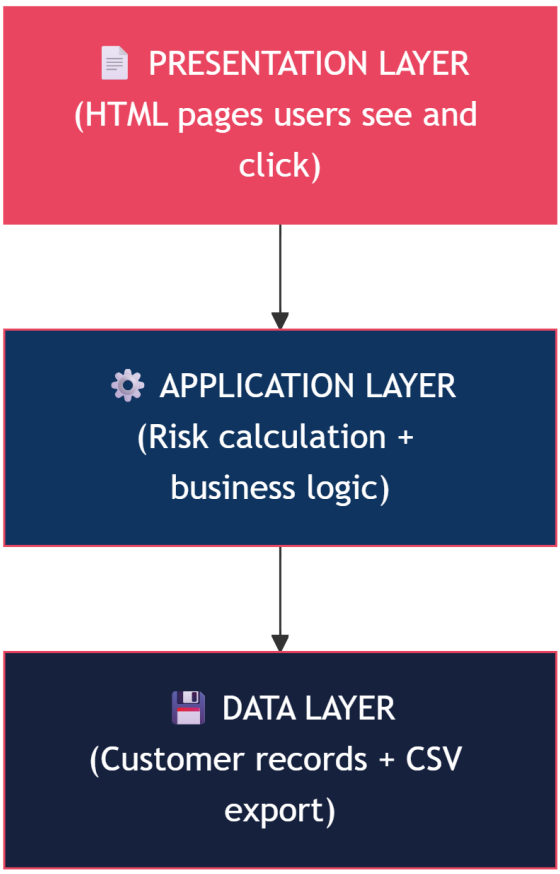
User Requirements (Who uses it):

- Financial Analyst: View scores and understand breakdown
- Bank Manager: See risk distribution, identify high-risk customers
- Evaluator: Verify calculations and documentation

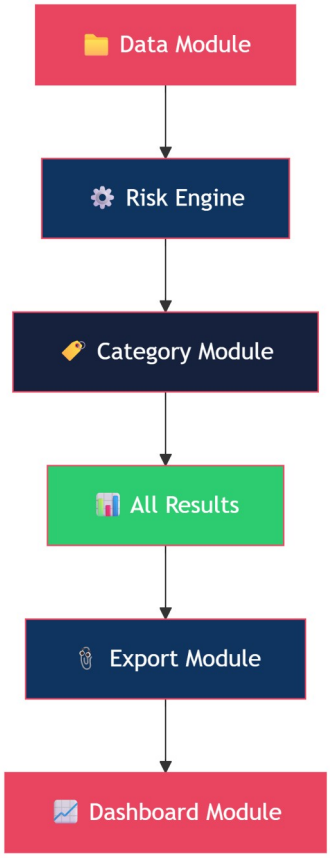
Environmental Requirements:

- OS: Windows, Mac, or Linux
- Python 3.8+
- 4GB RAM minimum
- No internet required

## Design and Architecture - Overview

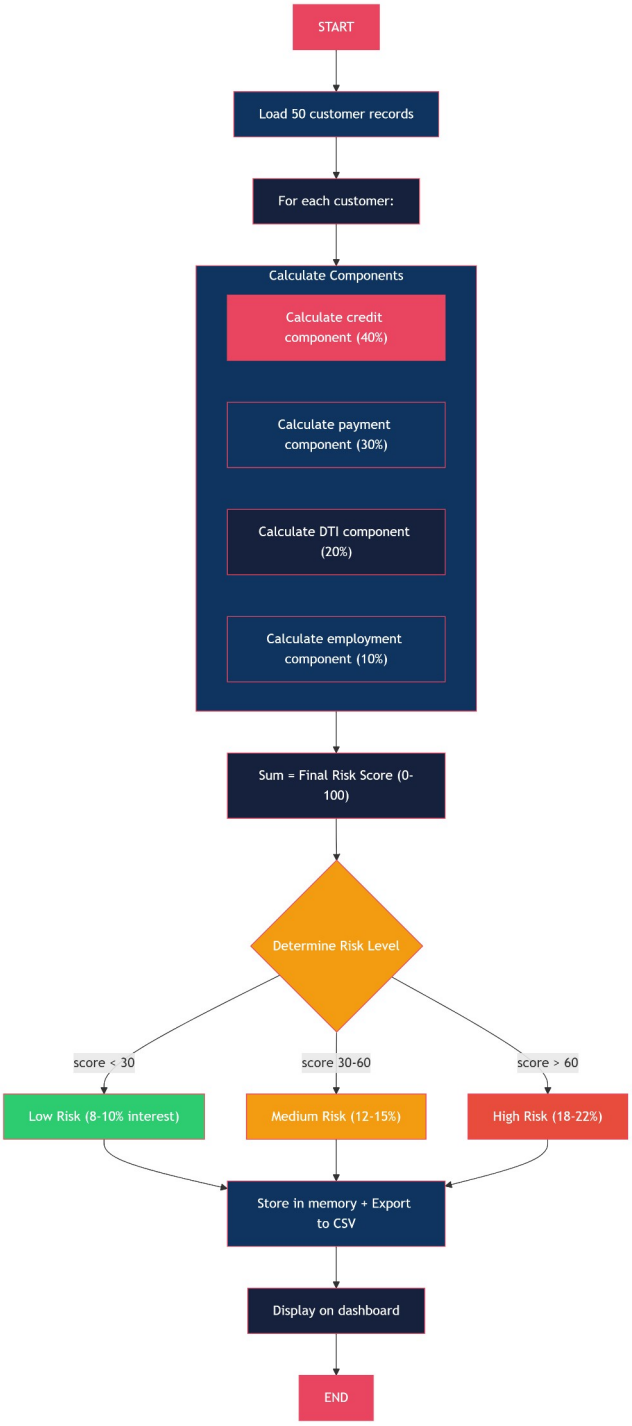


# Component Architecture

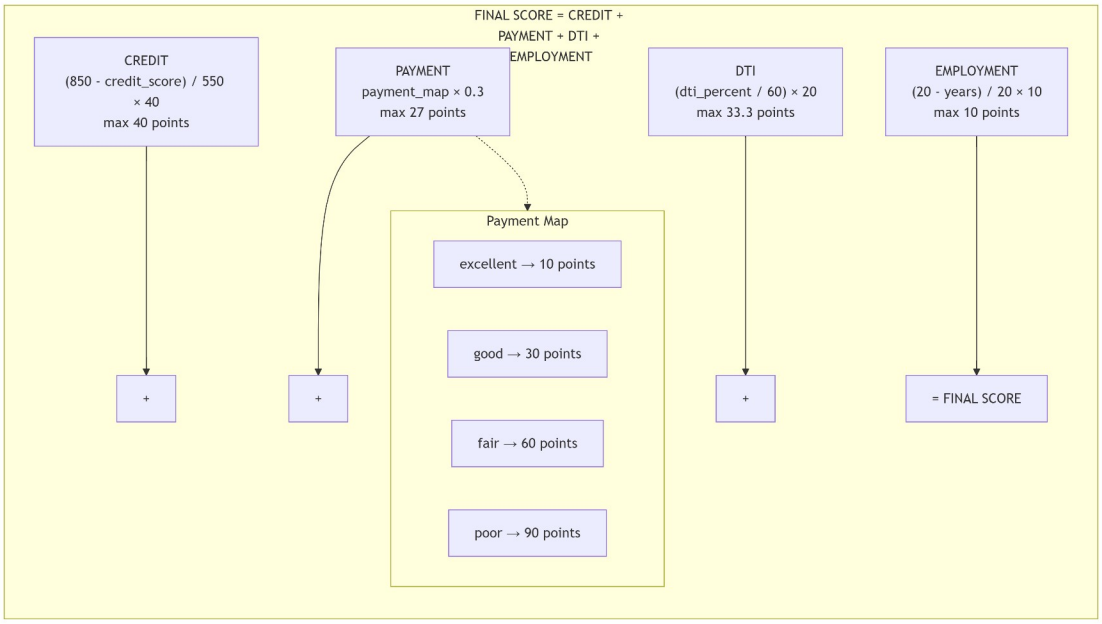



| Component | Job |
|---|---|
| Data Module | Store 50 Customer records |
| Risk Engine | Applies weight, calculate score |
| Category Module | Classifies Low/Medium/High |
| Export Module | Save to CSV files |
| Dashboard | Shows web pages |

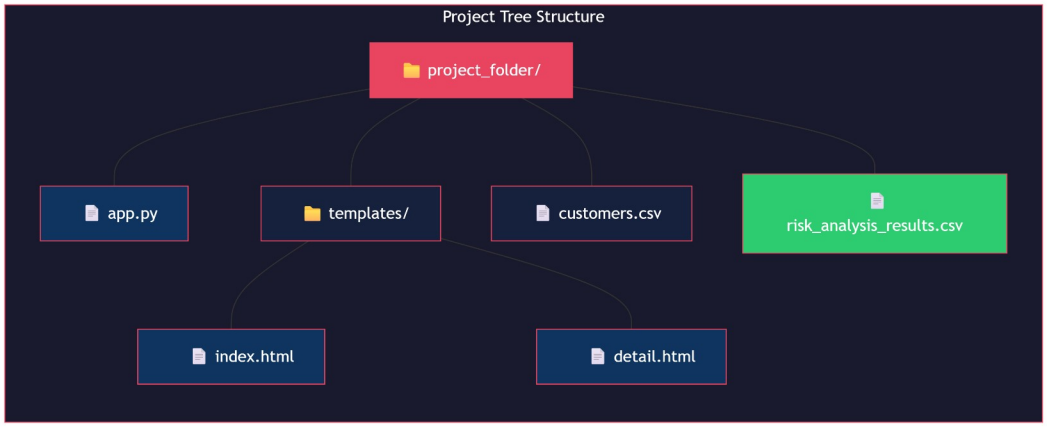
# Data Flow


START
Load 50 customer records
For each customer:
Calculate Components
Calculate credit component (40%)
Calculate payment component (30%)
Calculate DTI component (20%)
Calculate employment component (10%)
Sum = Final Risk Score (0-100)
Determine Risk Level
score < 30
score 30-60
score > 60
Low Risk (8-10% interest)
Medium Risk (12-15%)
High Risk (18-22%)
Store in memory + Export to CSV
Display on dashboard
END

# Risk Calculation Formula




# Implementation - Files Structure



Code organization in app.py:

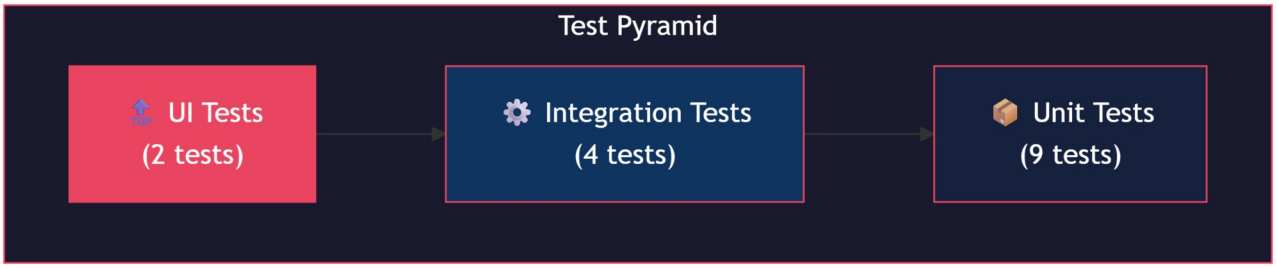
```
# Data section
customers_data = [50 records]

# Risk calculation
def calculate_risk(customer):
    # 40% credit + 30% payment + 20% DTI + 10% employment
    return score, category, rate

# Routes
@app.route('/')              # Dashboard
@app.route('/customer/<id>') # Detail page
```

## Implementation - Key Functions

Simple arithmetic, no complex libraries, easy to test.

```
def calculate_risk(customer):
    # Credit component (40%)
    credit_risk = (850 - customer['credit']) / 550 * 40

    # Payment component (30%)
    payment_map = {'excellent':10, 'good':30, 'fair':60, 'poor':90}
    payment_risk = payment_map[customer['payment']] * 0.3

    # DTI component (20%)
    dti_risk = (customer['dti'] / 60) * 20

    # Employment component (10%)
    emp_risk = max(0, (20 - customer['employment']) / 20) * 10

    # Total
    total = credit_risk + payment_risk + dti_risk + emp_risk

    # Category
    if total < 30: category = "Low Risk"
    elif total < 60: category = "Medium Risk"
    else: category = "High Risk"

    return round(total, 1), category
```

# Testing - Overview




| Type | What we tested | Result |
|---|---|---|
| Unit | Individual formulas | All pass |
| Integration | Full pipeline | 50/50 processed |
| UI | Pages load correctly | Dashboard works |
| Edge | Boundary values | Handled properly |

# Testing - Unit Test Results

```
Test 1: Credit calculation
Input: credit = 720
Expected: (850-720)/550×40 = 9.45
Result: 9.5 (rounded)

Test 2: Payment mapping
Input: payment = "excellent"
Expected: 10 × 0.3 = 3.0
Result:  3.0

Test 3: DTI calculation
Input: dti = 25
Expected: (25/60)×20 = 8.33
Result:  8.3
```

```
Test 4: Employment calculation
Input: years = 8
Expected: (20-8)/20×10 = 6.0
Result:  6.0

Test 5: Category Low Risk
Input: score = 25
Expected: "Low Risk", "8-10%"
Result: Pass

Test 6: Category Medium Risk
Input: score = 45
Expected: "Medium Risk", "12-15%"
Result:  Pass

Test 7: Category High Risk
Input: score = 75
Expected: "High Risk", "18-22%"
Result:  Pass
```

## Testing - Integration & Edge Cases

| Test | Description | Result |
|---|---|---|
| Full 50 customers | All processed without error | Pass |
| CSV export | File created with 50 rows | Pass |
| Dashboard loads | HTTP 200 response | Pass |
| Detail page load | Correct customer data shown | Pass |

Edge case handling:

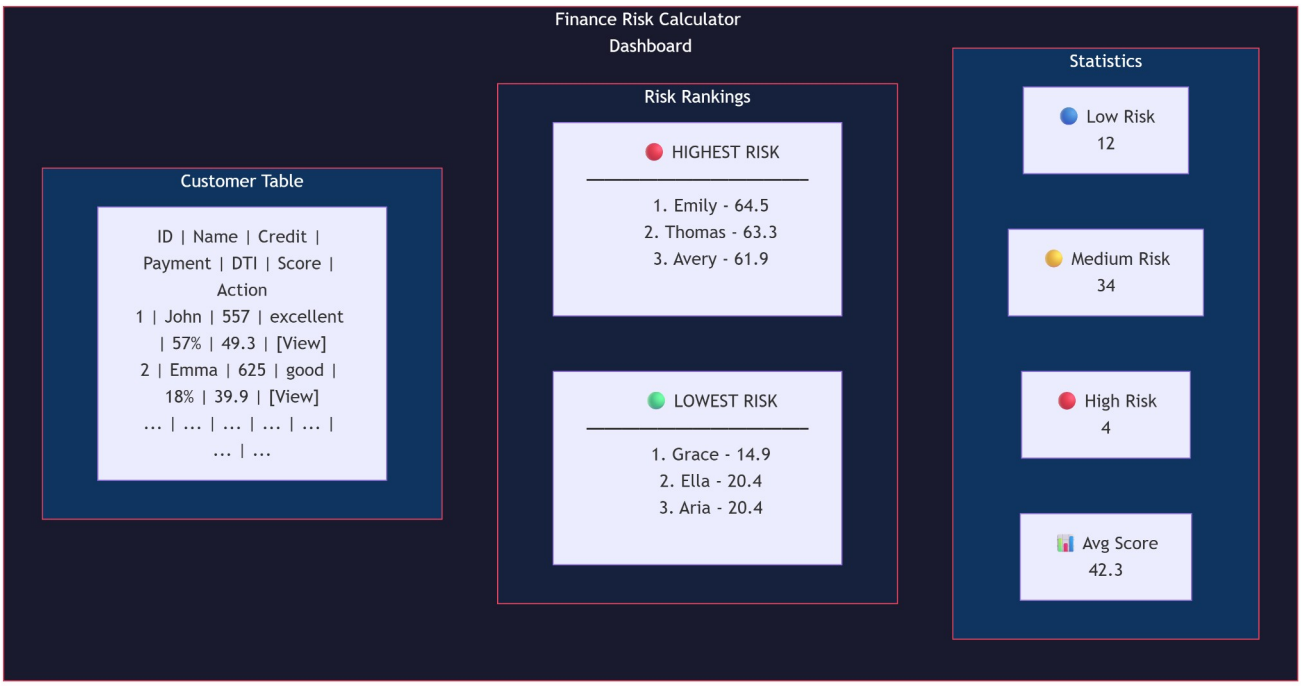
Edge case 1: Minimum credit score (300)
Result: credit_risk = (850-300)/550×40 = 40.0 (max)
Status: Handled

Edge case 2: Maximum DTI (100)
Result: dti_risk = (100/60)×20 = 33.3 (capped)
Status: Handled

Edge case 3: Zero employment years
Result: emp_risk = (20-0)/20×10 = 10.0 (max)
Status: Handled

Edge case 4: Customer not found (id=999)
Result: 404 error page
Status: Handled

# Graphical User Interface - Dashboard

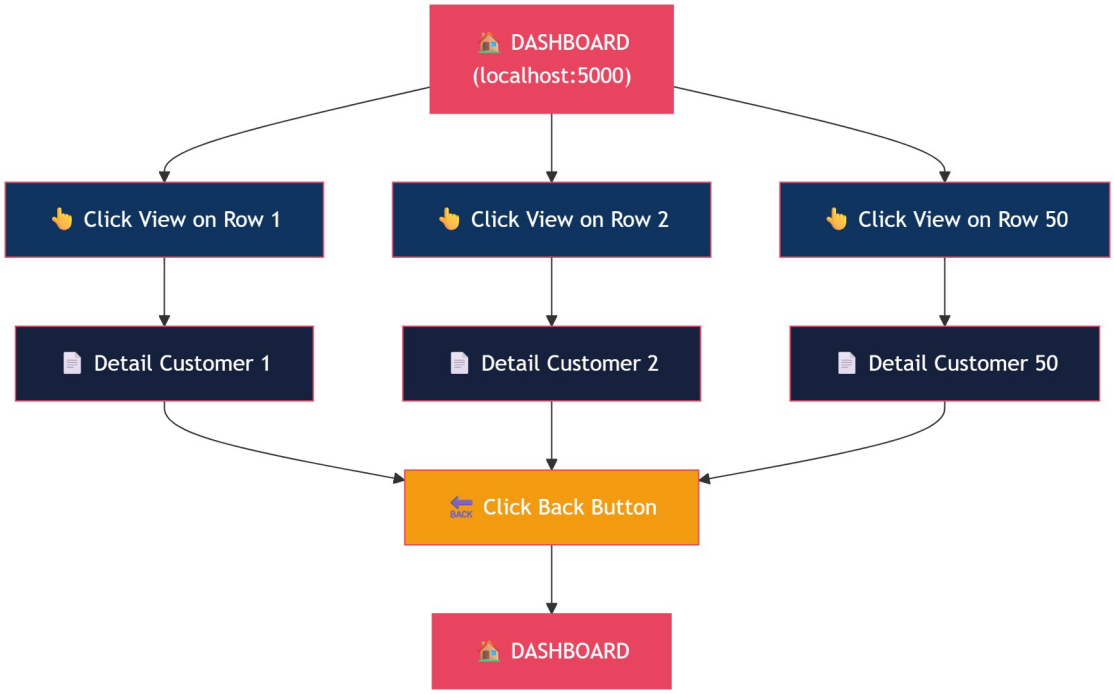


**Features:**

- Statistics cards show risk distribution
- Two lists show top 5 highest and lowest risk
- Table shows all 50 customers
- Each row has a "View Details" button

## Graphical User Interface - Detail Page

# UI Navigation Flow

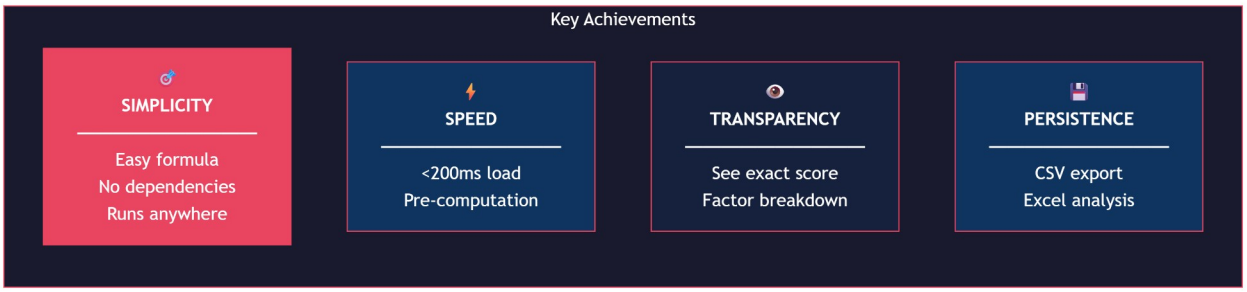



# Customer Testing

| User Type | Feedback | Action Taken |
|---|---|---|
| Financial analyst student | Easy to Understand | Kept same design |
| Bank manager | Risk colors help quick decision | Kept color coding |
| Professor | Formula is transparent | Added formula display |

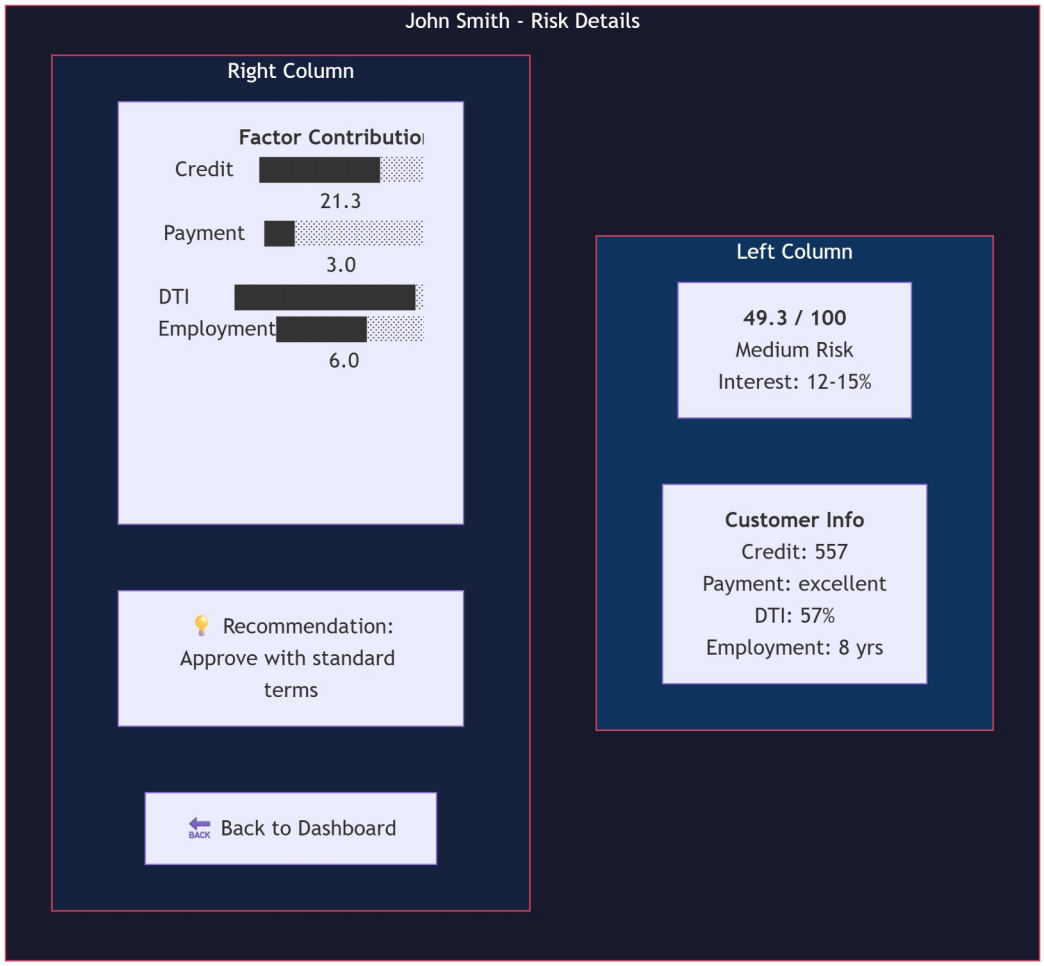
# Evaluation - Success Criteria

# Evaluation - What Worked Well



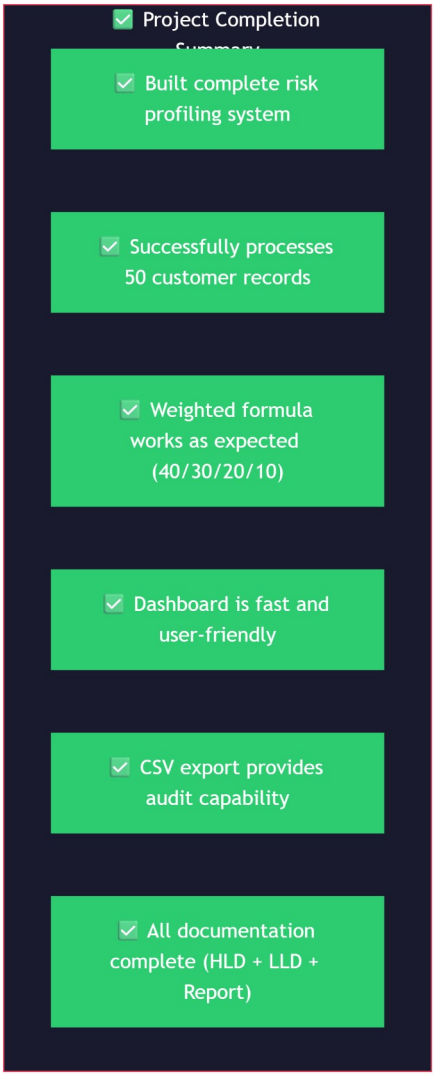

# Snapshots - Dashboard

# Snapshots - Detail Page




# Snapshots - CSV Export

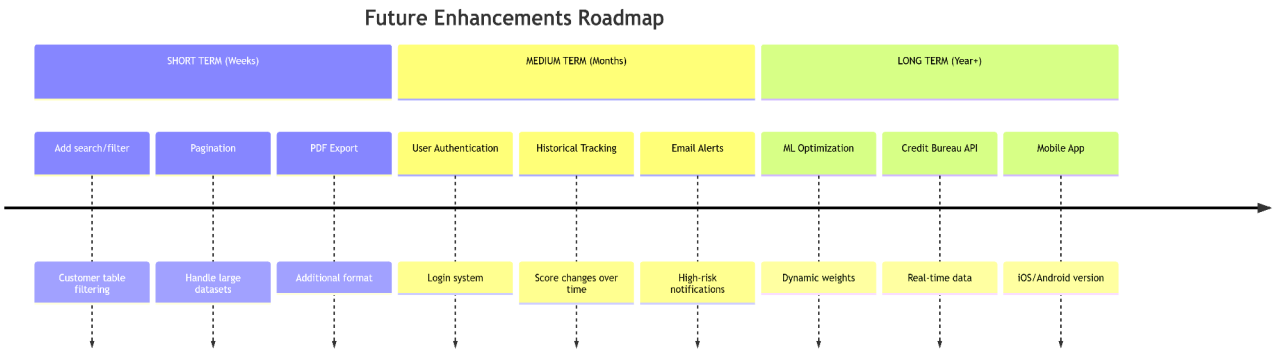
# Conclusions




Key learning: Risk profiling is mathematics. A simple weighted formula can effectively rank customers by risk level. Transparency is important - users should understand why they got a particular score.

The system demonstrates exactly how real credit scoring engines work.

# Further Development




# References

Technology documentation:

- Flask Documentation: https://flask.palletsprojects.com/
- Jinja2 Documentation: https://jinja.palletsprojects.com/
- Python CSV Module: https://docs.python.org/3/library/csv.html

Financial domain references:

- FICO Score Range: https://www.fico.com/
- Debt-to-Income Ratio: Consumer Financial Protection Burea

Code files:

- app.py - Main application
- index.html - Dashboard template
- detail.html - Detail page template
- risk_analysis_results.csv - Output file

# Appendix

Project Timeline Summary

The project was completed over a period of approximately 12 weeks. The first two weeks were dedicated to the project proposal, where requirements were gathered and the initial scope was defined. Weeks three and four focused on creating the High-Level Design document, which outlined the system architecture and component interactions. Weeks five and six were spent on the Low-Level Design, detailing class structures, sequence diagrams, and implementation specifications. The actual coding and implementation took place during weeks seven and eight, where the Flask application, HTML templates, and risk calculation logic were written. Weeks nine and ten were reserved for testing, bug fixes, and validation of all 50 customer records. The final two weeks, weeks eleven and twelve, were used to prepare this Final Report and compile all documentation for submission.

Team Information

This was a solo project completed by the undersigned student. Industry mentorship was provided by [Mentor Name], who offered guidance on financial risk modeling and industry best practices. Faculty guidance was provided by [Faculty Name], who supervised the academic aspects including documentation standards and project evaluation criteria.

Code File Descriptions

The project consists of four main files. The first file is app.py, which contains the Flask application, route definitions, risk calculation function, and CSV export logic. This is the main file that must be executed to run the application. The second file is index.html, located in the templates folder, which defines the dashboard page layout including statistics cards, risk lists, and the customer table. The third file is detail.html, also in the templates folder, which defines the customer detail page showing factor breakdowns and recommendations. The fourth file is risk_analysis_results.csv, which is generated automatically when the application starts and contains all 50 customer records with their calculated risk scores and categories.

How to Run the Project

To run the project, the user must first ensure Python 3.8 or higher is installed on their system. Then, open a terminal or command prompt in the project folder and run the command pip install flask to install the Flask web framework. After Flask is installed, run the command python app.py to start the server. The terminal will display a message indicating that the server is running on http://127.0.0.1:5000. The user can then open a web browser and navigate to this address to access the dashboard. To stop the server, press Ctrl+C in the terminal.

## Known Limitations

The system has a few limitations worth noting. First, customer data is hardcoded and cannot be modified through the user interface without editing the source code. Second, the system does not include user authentication, so anyone with access to the URL can view all customer data. Third, the risk weights (40%, 30%, 20%, 10%) are fixed and cannot be adjusted by the user without modifying the code. Fourth, the system is designed for a maximum of 50 customers; processing larger datasets would require code changes and may impact performance. These limitations are acceptable for an academic demonstration but would need to be addressed for a production deployment.

## Submission Checklist

The following items are included with this submission: the High-Level Design document (HLD), the Low-Level Design document (LLD), this Final Report, the complete source code (app.py and templates), the sample CSV output file, and screenshots of the running application. All documents have been formatted according to the institute's guidelines and are ready for evaluation.

## Contact Information

For any questions regarding this project or access to the complete source code, the student can be reached at [Your Email Address]. The complete project is also available for demonstration upon request.